Chapter 2

State of the Art

Different programming languages have adopted various paradigms, including functional or logic models. Yet imperative programming remains a popular paradigm due to how popular languages have been designed. However, as applications shifted towards event-driven architectures, traditional imperative programming did not come without its drawbacks. Traditional imperative programming was simple. Each imperative program was code that defined its step-by-step control flow and state transitions. However, when an imperative programming language is used for an event-driven architecture, some problems may arise, such as 'inversion of control' or 'callback hell'. Inversion of control is a problem commonly associated with codebases using frameworks, where instead of the programmer calling functions provided by the framework, the framework calls the functions defined by the programmer.

```
func1(argument0, function(argument1){
    // compute result1
    func2(result1, function(argument2){
        // compute result2
        func3(result2, function(argument3){
            // compute result3
            func4(result3, function(argument4){
                //etc..
            })
        })
    })
})
```

Listing 3: Illustration of callback hell in JavaScript

Callback hell occurs when programmers ~~are required to~~ pass a callback function as an argument to another function, which calls the passed function with the result of its computation, yet that function also expects a function to be passed to continue its execution. While this is not necessary an issue if the nesting is one or two layers deep, it becomes 'hell' once it goes multiple layers deep ~~or ad infinitum~~, as shown in Listing 3. Reactive programming seeks to solve these issues.

2.1 Reactive Programming

Reactive programming (RP) is a declarative programming paradigm that allows the developer to declare a program in terms of dependencies, instead of having to manually manage these dependencies. The actual propagation of the state change in the dependency chain is abstracted away in the language design. For instance, in a program you could have two variables x and y that hold values and variables s and p which hold the sum and product of x and y respectively. Under the imperative paradigm, whenever x or y would change state, it would require manual

```
int x = 5;
int y = 10;
int s = x + y;
printf("%d", s) // 15;
x = 10; // state change
printf("%d", s) // 15;
// requires manual recomputation of s
s = x + y;
printf("%d", s) // 20;
```

Listing 4: Manual recomputation of s, a dependent on x

recomputation of s as shown in Listing 4. In reactive programming, it is sufficient to update either x or y and the change is propagated throughout its system automatically, as shown in Listing 5. In essence, it reacts to changes in its input values and automatically propagates it

```
int x = 5;
int y = 10;
int s = x + y;
printf("%d", s) // 15;
x = 10; // state change
// No manual recomputation needed
printf("%d", s) // 20;
```

Listing 5: Automatic recomputation of s, a dependent on x (Pseudocode)

through itself and updates its output values. These values are also called *signals*. There seem to be two main families in how reactive programming is implemented. One of which is functional reactive programming (FRP) and the other graph-based reactive programming. FRP languages such as E-FRP, FRPNow or Yampa are often implemented in a host language, such as, but not limited to: Haskell, Racket, Java, Scala or JavaScript [1]. These FRP languages then inherit the toolchains of the host language. However, there are also FRP languages are that are an independent language. They do not rely on a host language but instead have their own dedicated tools (compiler, interpreter, etc...). FRP Languages consist of lambdas or functions that are (re)-applied every turn in the reactive system.

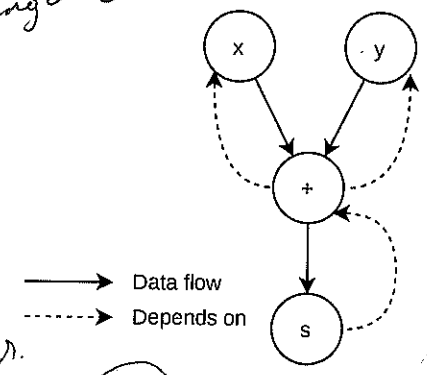


Figure 2.1: DAG for Listing 5

Graph-based reactive programming languages compile the program into a directed acyclic graph (DAG), which is also known as the dependency graph of the program. The DAG describes the data flow between different components within the program, as an example, we can construct the DAG of Listing 5. Since s depends on the result of the sum, and the sum depends on x and y, we can construct the DAG as shown in Figure 2.1. Here we can see that x and y flow to the sum, whose result flows to s. Examples of graph-based reactive programming languages are REScala, FrTime and Flapjax.

A reactive application created with an RP language built on top of a host language usually consists of two parts. A reactive part of code, which is a 'reactive' extension of the host language and a non-reactive part of the code. The non-reactive part of the code are features that are made available from the host language through a mechanism known as *lifting* [3]. However, lifting can lead to issues such a diverging code, which leads to unresponsive programs or a mismatch between the reactive and non-reactive code, which leads to faulty behaviour. The SOFT lab at the VUB proposes a novel reactive programming language called **Haai** [3] (Pronounced 'high') that is meticulously designed to be reactive-only, avoiding these issues completely.

```
(defr (sum-product x y)
  (def s (+ x y))
  (def p (* x y))
  (out s p))
```

Listing 6: Reactive program in Haai

Haai uses reactors as its only computational abstraction for declaring a reactive program. Which is a declarative way of defining signals and their dependencies. A Haai program consists of reactor definitions (defined using *defr*) and reactor deployments (similar to application expressions in Racket). An example of such a Haai program can be seen in Listing 6. Here we can see that the reactor reacts to the input signals x and y, and computes their sum and product by deploying the $*$ and $+$ reactor with the input signals and outputs the result of the deployments. Upon changing the values of the input signals, the reactor will automatically recompute their dependant output values.

2.2 RP languages targeting MCUs

In Section 2.1 we discussed reactive programming. Reactive programming has many applications, from web applications that react to the input of users to embedded devices, which react to signals from hardware. In this thesis we want to focus on RP related to embedded devices, whose usage has increased due to their wide variety of applications, such as on the internet of things (IoT), edge computing and artificial intelligence (AI) [9], [11].

As previously mentioned, many RP languages are built on top of host languages which often require the toolchains of those host languages to run. However, in environments where resources are constrained such as on microcontrollers, it is hard to ship an RP program created in Yampa. This is due to the fact that Haskell runtimes are very large and do not fit the resource constraints on microcontrollers. Despite this challenge there have been efforts to get RP onto microcontrollers.

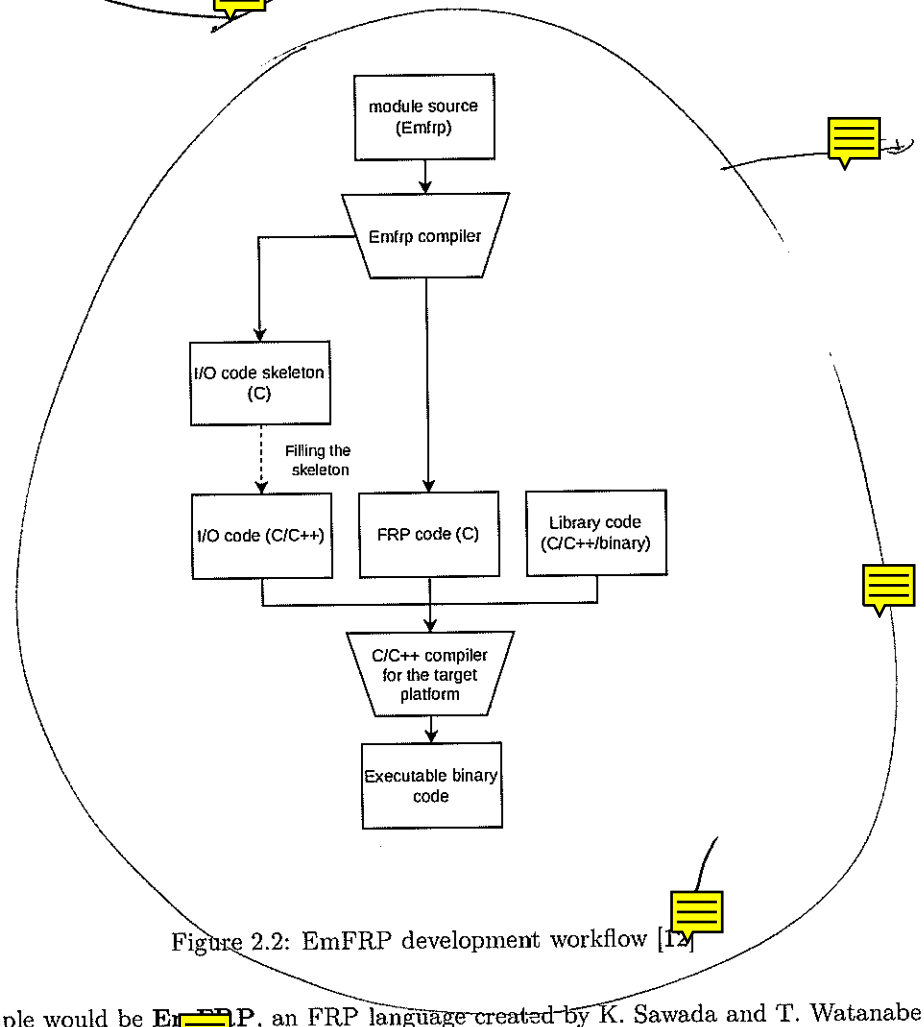


Figure 2.2: EmFRP development workflow [12]

An example would be EmFRP, an FRP language created by K. Sawada and T. Watanabe

that mainly targets small-scale embedded systems [12]. Like Haai, EmFRP is a graph-based RP language. This means that EmFRP follows the same philosophy in terms of values that depend on other signals and automatic change propagation. More concretely, a EmFRP program is written in terms of nodes, who may refer to other nodes that it depends on. Conceptually it works the same as the DAG that Haai generated in Figure 2.1.

However, since EmFRP is designed to work on small-scale embedded systems, some design considerations have been made. For instance, the dependency graph must be known statically, meaning it cannot change during runtime. Additionally, it does not allow recursion in function or in type definitions and nodes must always have a name. This because they are not first class values in the language.

```
module Thermostat
in
    temperature : Int,   # temp sensor
    pulse1min : Bool     # periodical pulse
out
    switching : Bool     # heater state
use
    Std                  # standard library

node init [ False ] switching =
    if pulse1min then
        temperature < 50
    else
        switching@last   # previous value
```

Listing 7: Thermostat controller in EmFRP [12]

```
#include "Thermostat.h"
void Input(int *temperature, int *pulse1min){
    /* Your code goes here ... */
}

void Output(int *switching){
    /* Your code goes here ... */
}

int main(){
    ActivateThermostat();
}
```

Listing 8: I/O Code Skeleton for Thermostat [12]

Furthermore, interacting with hardware requires using magic constants such as memory addresses, that are architecture- or peripheral specific. To address this, the EmFRP compiler generates an I/O code skeleton to connect the program with the hardware. The developer remains responsible for completing the skeleton code through.

The way EmFRP works is as follows: The developer creates a series of EmFRP modules, the

EmFRP compiler then generates I/O skeleton code in C and FRP code in C. The developer is then required to fill in the I/O skeleton code. A schematic of the workflow can be seen in Figure 2.2.

In the paper introducing EmFRP, they show an example with a thermostat, where each minute, the temperature is read and depending on the temperature value, the heater is turned on or off. The code for this example can be seen in Listing 7. The EmFRP program is then compiled into the I/O skeleton shown in Listing 8 as well as the FRP code in C. Once the skeleton code is completed, the program can be further compiled by the relevant C compilers into the required binaries.

Another RP language targeting embedded devices is CFRP [13]. By the same creators of EmFRP, K. Sawada, T. Watanabe, as well as K.Suzuki and K. Nagayama, CFRP was created as a pure FRP language that compiles to stand-alone C++ code. It was created to show that FRP is beneficial for developing software for small-scale embedded devices.

```
-- Including C++ header files
%{
#include "sensors.h"
#include "ac_ctrl.h"
%}

-- Declaring identifiers connected to external devices
%input tmp :: Signal Float = sen_tmp;
%input hmd :: Signal Float = sen_hmd;
%import ac :: Signal Bool → Signal () = ctrl_ac;

-- Main expression
let di t h = t - 0.55 * (1 - 0.01 * h) * (t - 14.5) in
ac (lift1 (\x → x > 24) (lift2 di tmp hmd))
```

Listing 9: Example program in CFRP [13]

It was designed as follows: A CFRP program consists of a series of declarations, which denote signals and links to hardware (in C++). Followed by a single expression that builds to reactive system. An example can be seen in Listing 9 where the air conditioner is turned on or off depending on a discomfort index. The discomfort index is calculated from the temperature (tmp) and humidity (hmd). Notably it seems to use the same 'base' datatypes that C++ provides but signals get an additional annotation (the Singal type). This is to make the datatype predictable at compile time.

Furthermore, it uses a single Haskell-like expression to construct a dependency graph, just like EmFRP and Haai do, which is then used to compile to the C++ program. The caveat with CFRP is that C++ objects and functions referred to in the CFRP file have to be declared in separate C++ files. This means that only the 'global' reactive system is declared in CFRP and the rest in C++.

Whilst there are more RP languages targeting embedded systems like Atom [14] or Copilot [15]. They seem all to compile to a low-level programming language such as C, C++, Rust or Zig. Though the more customary languages that these RP languages compile to are

either C or C++.

At the VUB a virtual machine (VM) was created for Haai to be able to run Haai on the bare metal. The VM is called Remus [7], which was later re-implemented in Rust and with some modifications a program was created to run a Haai program on a M5Stick [8]. The benefit of maintaining a s-expr-like syntax with dynamic typing remains, at the cost of having to compile the Haai program to Remus bytecode. Which can then be interpreted by the Rust based implementation of Remus.

At this point one could ask him or herself what the point is, of examining these RP languages that target MCUs. Well as mentioned in Section 2.2, due to the increase in use of microcontrollers. Though in this thesis we want to focus more on a system of these embedded devices. Take a robot for example: A robot has sensors as input and has other peripherals that are then used to interact with the real world, depending on these inputs. In other words, the robot reacts to the sensors by instructing the motors what to do or where to move next.

To build this system of embedded devices we could create some reactive programs in an RP language of choice that targets these devices, but this would leave the developer responsible for implementing all the communication between the devices as well as some central computing unit that coordinates all these devices and peripherals. Most of the time requiring the developer to use a library that uses some kind of communication protocol under the hood (Bluetooth, UART, I2C, SPI, USB, etc...). Easier would be if we could use some kind of standardized toolkit that saves the developer from these complexities, which is what we will be looking into next.

2.3 Systems to program robots

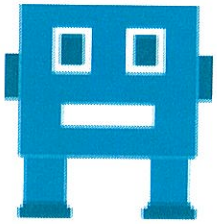
In this section we are going to have a look at the following systems that aid development in robotics: Open Robots Control Software (OROCOS) [5], Robot Operating System (ROS) [4], [16], [17], [18] and Yet Another Robot Platform (YARP) [6].



(a) OROCOS logo



(b) ROS logo



(c) YARP logo

Figure 2.3: Robotics platforms

2.3.1 OROCOS

OROCOS, started at the Katholieke Universiteit Leuven (KUL), as a project aimed at becoming a general-purpose and open robot control software package. On a more technical side of things, it is software for all sorts of robotic devices and computer platforms (operating systems). OROCOS features software components for kinematics, dynamics, planning, sensing, control, interfacing, etc... through a middleware. The system also provides features in the form of modules that can be split up in the following categories:

- Supporting modules
  Software that does not interact with the hardware directly but rather supports the interaction with it. These modules can range from simple mathematical modules to aid vector operations, configuration modules, simulation modules to real-time operating system modules.

- Robotics modules. These are the modules specifically interacting with the hardware such as kinematics and dynamics modules or servo controller modules. These robotics modules tend to use one or more of the supporting modules to make the usage or development of these robotics modules easier.

- Components. These are the Common Object Request Broker Architecture (CORBA) with their Interface Definition Language (IDL) descriptions. These components are built upon the previous modules and are used as the building blocks of robots, as they allow for multi-machine inter-process communication.

~~Unfortunately~~ OROCOS does not allow it to be used on the bare metal. Meaning it is only usable on a system that has an OS. However, with the first version of ROS, the real-time toolchain (RTT) of OROCOS, called OROCOS RTT was often used alongside ROS. Nowadays, the second version of ROS, or ROS 2 is used in favour of ROS + OROCOS RTT, or the OROCOS Component Library (OCL) as ROS 2 absorbed both the toolchain and component features of OROCOS and thus no longer needs them as middleware. The hardware libraries for Kinematics Dynamics (KDL) or Bayesian Filtering (BFL) are still used. OROCOS also provides support to interop with other robotics systems such as ROS and YARP. To summarize, OROCOS seems inadequate as a robotics system to build a distributed system of embedded devices because it does not run on the bare metal. Additionally, it is not used as a communication layer across devices and actually depends on other robotics systems to bridge the communication such as the next system. OROCOS could however, be used as a supporting tool, depending on the domain of robotics.

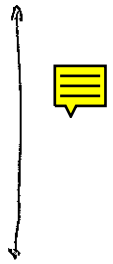
2.3.2 ROS

Originating from Stanford, Robot Operating Systems, or ROS, is in fact not an operating system but rather a Software Development Kit (SDK). Like OROCOS, ROS provides modules to aid in the development of robotics systems. The modules can be categorized as follows:

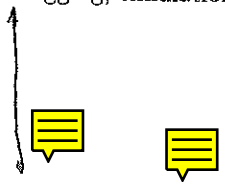
- Middleware: This encompasses communication among components, from network APIs to message parsers.



- Algorithms: Commonly used algorithms when building robotics applications



- Developer tools: A suit of CLI tools for configuration, launch, introspection, visualization, debugging, simulation and logging to name a few.



The middleware is essentially a structured communication layer above the operating systems of a compute cluster [16]. A ROS cluster consists of several on-board devices and off-board devices, which can connect and disconnect at runtime. Consequently, the designers of ROS chose for a peer-to-peer (p2p) system as having a central server would mean a lot more network saturation as messages would have to a central point and would then have to be forwarded to their respective recipients. In a p2p system each message would be forwarded to an intermediary recipient who has the address of the final recipient or another intermediary, leading to less network saturation and shorter distances travelled.

The philosophy of ROS of using multiple standalone components within its system allows for developers to reuse or repurpose existing hardware libraries within ROS, but without actually depending on ROS. This also allows for robust testing of these libraries, without being dependent on ROS. The decoupling of these components also means that they can be used in other robotics systems where there is no inherent dependency and vice versa, giving ROS access to existing standalone components, such as drivers and vision algorithms.



As previously mentioned, the designers of ROS chose for a p2p system. However, the actual implementation is divided into nodes, messages, topics and services. Nodes are processes that perform computations. This means that each device in the system could consist of one or more nodes. Since these nodes also communicate with each other, we could create a graph with the edges denoting where the nodes send their messages to.



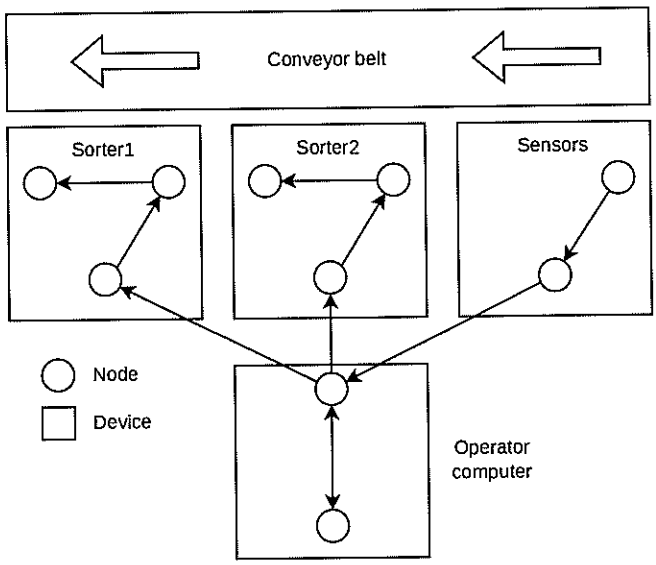


Figure 2.4: ROS node graph example

Figure 2.4 shows a robotics example where some products are being transported down a conveyor belt. However, these products have to be classified by colour, which the sensors on the 'sensors' device can detect. The sensors can then send the detected colour to the operator computer, which selects the appropriate sorter to sort the product. Each sorter consists of an arm and a sensor. The sensor detects when the product is in the sorting range and sends a signal to the arm, which sorts it. Furthermore, the operator computer shows the detected colours on the screen and allows the user to manually operate a sorter if need be thus completing the graph shown in the figure.

```
fieldtype1 fieldname1
fieldtype2 fieldname2
fieldtype3 fieldname3
```

Listing 10: IDL example (pseudocode)

Messages that these nodes send to each other are actually strictly typed datastructures. These can consist of the primitive types (integer, floating point, boolean, etc...), arrays of primitive types as well as constants though they must be defined in a `.msg` file, or an interface definition language (IDL). An example can be seen in Listing 10. Messages can also be composed of other messages or arrays of messages which can be nested arbitrarily deep. Do note that the types used in the IDL depend on the language where the ROS library is used. If it is used in C++ you can use the native C++ primitive types. In Python, you would have to use the library-provided types that map to their respective C++ types. The ROS Client library that

provides the API to create a ROS graph is called rcl, which is written in pure C. Depending on the language used, the API is built upon further resulting in rclcpp (C++), or rclpy (Python).

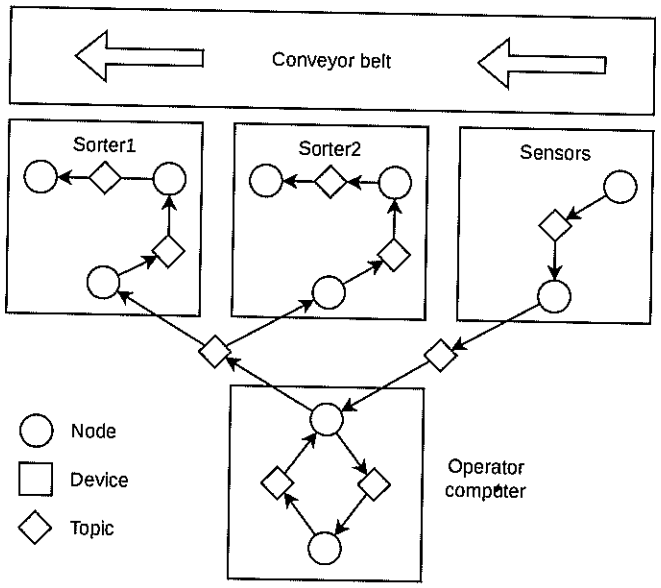


Figure 2.5: ROS node graph with topics

To be able to broadcast messages asynchronously to multiple devices, ROS sends messages through a topic. Nodes subscribe to a particular topic if they wish to receive messages published to the topic. This allows for a many-to-many relationship within the ROS node graph. A re-illustration of Figure 2.4 can be seen in Figure 2.5, this time illustrated with the topics, where unidirectional one-to-one, one-to-many, many-to-many or many-to-one relationships are allowed. A bidirectional relationship between two nodes can be created by making one of the nodes a subscriber to a topic and the other a publisher as well as the inverse on another topic as seen on the operator computer.

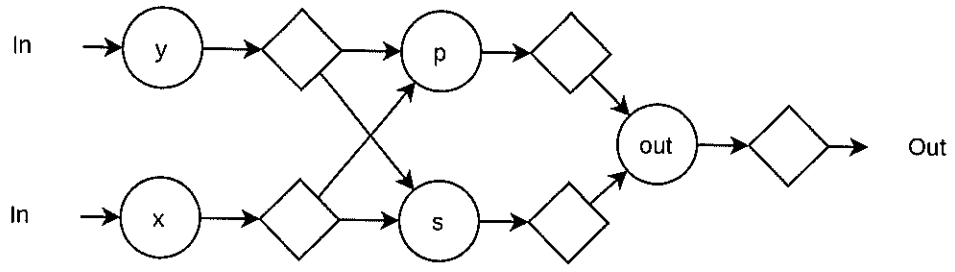


Figure 2.6: Product sum in a ROS node system

Interestingly, we can use topics to create a graph-like structure, just like the dependency graphs that are generated from an RP program in Haai, EmFRP or CFRP. Schematically we can recreate the DAG defined in Figure 1.1 in a ROS system, as shown in Figure 2.6.

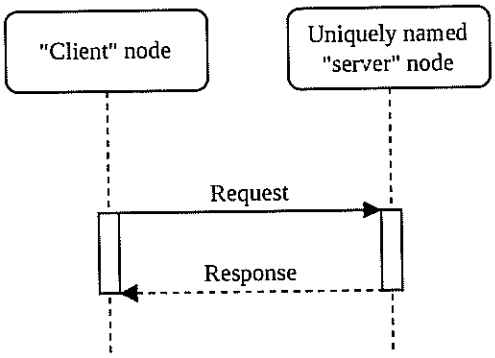


Figure 2.7: ROS service protocol

Whilst topics are the main communication protocol that ROS uses, it is lacking for synchronous communication or short-lived communication that expects a response from a request. To solve this, the designers implemented services, which works analogous to the web services. A node places a request to a uniquely named node which returns a response after processing the request, just like web services. Like topics, service definitions must be stored in a `.srv` file.

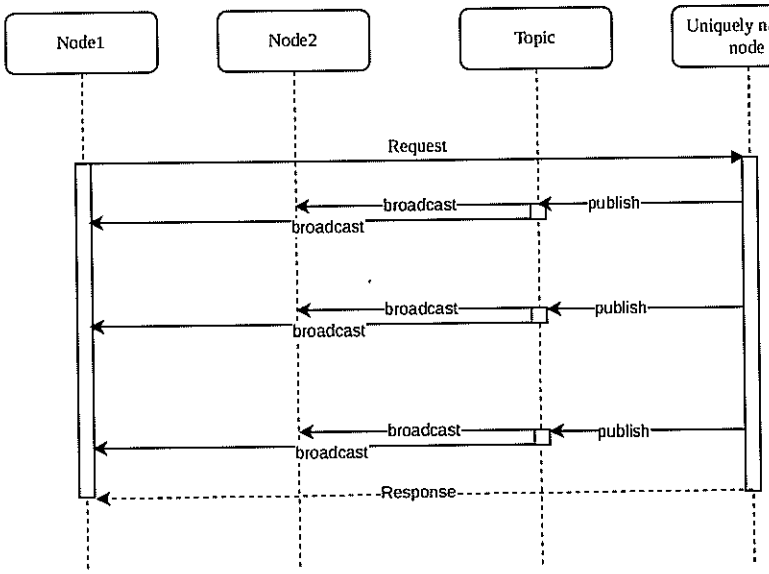


Figure 2.8: ROS actions protocol

In case neither topics nor services are inadequate and, you need the best of both worlds there is also actions. Which is a combo of both topic and requests and ideal for long-running tasks and works as follows: It starts off like a service, a node sends a request to a uniquely named node, which in turn has to process the request. During the processing of the request the uniquely named node produces some feedback which gets published to a topic and is sent to its subscribers. After processing the request a response is sent back to the node that made the request. Unlike services or topics, actions allow for cancellation of the request if needed. Like services and topics, actions must be defined in an `.action` file.

At the end of the previous section we mentioned ROS 1 and ROS 2. Essentially ROS 1 is the predecessor of ROS 2 as the number probably already suggested, however they are vastly different. As ROS 1 grew in popularity across robotics, its limitations became increasingly evident. Security, reliability in nontraditional environments, and support for large-scale embedded systems became essential [17]. Since ROS 1 did not provide solutions for these concerns, companies were forced to create workarounds. To solve these issues they rebuilt ROS from the ground up and thus ROS 2 was born. A summary of the comparison of features between the versions can be seen in Table 2.1. Without getting in to too much of the technical

Category	ROS 1	ROS 2
Network transport	Bespoke protocol built on TCP/UDP	Existing standard (DDS), with abstraction supporting addition of others
Network architecture	Central name server (roscore)	Peer-to-peer discovery
Platform support	Linux	Linux, Windows, and macOS
Client libraries	Written independently in each language	Sharing a common underlying C library (rcl)
Node versus process	Single node per process	Multiple nodes per process
Threading model	Callback queues and handlers	Swappable executor
Node state management	None	Lifecycle nodes
Embedded systems	Minimal experimental support (rosserial)	Commercially supported implementation (micro-ROS)
Parameter access	Auxiliary protocol built on XMLRPC	Implemented using service calls
Parameter types	Type inferred when assigned	Type declared and enforced

Table 2.1: Summary of ROS 2 features compared with ROS 1 [17].

changes, the most noteworthy changes would be the new underlying network communication, Data Distribution Service, which is also used in critical infrastructure. The peer-to-peer discovery and the support for embedded systems.

The change to DDS as its network communication solved much of the problems as DDS has best-in-class security, embedded and real-time support, multi-robot communication, and operations in non-ideal networking environments [17]. Peer-to-peer discovery is also one of its core defining features, as previously discussed.

Micro-ROS

ROS now also supports embedded systems. It does so because a robot is assumed to have sensors, actuators or other peripherals. These robots could contain microcontrollers that need to communicate with CPU(s) running ROS 2. ROS 2 supports embedded systems by providing a dedicated SDK called micro-ROS [18] for resource constrained devices (such as microcontrollers). The SDK provides all major features that the ROS 2 SDK provides by reusing as many packages as possible and providing them in a C by extending the existing rcl libraries with the rclc packages.

To be able to provide all these major features, micro-ROS relies on an open source implementation of DDS for eXtremely Resource Constrained Environments [19] (XRCE or XRCE-DDS) by eProsima called micro XRCE-DDS [20]. Micro XRCE-DDS comes with an agent that bridges the gap between XRCE-DDS and standard DDS, allowing microcontrollers to communicate with CPU(s) running ROS 2 seamlessly, independent of the underlying communication mediums (UART, I2C, Bluetooth, Serial, etc...).

Unfortunately, micro-ROS cannot be used on every single microcontroller. As ROS 2 was initially developed for POSIX based operating systems, some clock and timing functions remain of the essence. This means that the microcontroller has to either be able to run a RTOS providing these functions or have abstractions providing them. Currently micro-ROS supports some Real-Time Operating Systems, such as FreeRTOS, Zephyr and NuttX, as well as some boards such as Espressif ESP32, Raspberry Pi Pico, some arduino boards and some STM boards. There is also experimental support for bare-metal usage, other RTOS'es, and other boards but these often have to use a bridge providing the necessary abstractions, and it is not always guaranteed to work due to hardware limitations.

Whilst micro-ROS tries to provide as many of the features that ROS 2 provides, there are some considerations that have had to be made, seeing as the environment has resource constraints.

- The messages interchanged between two nodes, including at least one micro-ROS node must be a fixed size. This is to avoid dynamic memory allocations, which is heavily discouraged to do on a microcontroller post build times. This is because it is a best practice to allocate all the memory you need upon initialization, as dynamic memory allocations during runtime could yield undefined behaviour when one runs out of memory to allocate.
- To deal with different mechanisms and scheduling APIs of the underlying RTOS, rclc provides a flexible Executor on the level of individual callbacks.
- ROS 2 provides a launch system to start up the nodes of the ROS node graph. Micro-ROS does not have this due to different underlying RTOS mechanisms and leaves it up to the developer.
- Standard ROS 2 allows for dynamic loading of nodes into running processes. Since the program has to be flashed onto the microcontroller, the node composition of the micro-ROS program has to be known ahead of time, and thus at compile-time.
- Due to hardware limitations, most microcontrollers are unable to handle cryptographic libraries, leading to DDS-XRCE not being secure by default and instead delegating this

responsibility to the underlying transport medium. Since a microcontroller running micro-ROS talks to one micro-ROS agent on a PC, the PC running ROS 2 can then encrypt the message since it uses DDS-security. It leaves to check if the underlying transport medium from microcontroller to PC is safe. Since eavesdropping wireless transport is trivial, all wireless transport mediums (eg: Bluetooth) encrypt packets over the air. Wired transport mediums are inherently safe, as they are physically contained, though some mediums do run some type of encryption.

To summarize, ROS is an SDK which provides modules to aid robotics development. Most notably is their communication layer which allows for peer-to-peer communication. ROS also provides an SDK for development on microcontrollers, though not all microcontrollers. Since we can leverage its communication layer to form an RP dependency graph-like structure, we could in theory convert an RP program into a ROS program, as well as send parts of the RP program to microcontrollers, using the micro-ROS SDK.

2.3.3 YARP

Let us talk about Yet Another Robotics Platform or YARP. Created for and by researchers in humanoid robots, YARP is yet another robotics platform aimed for long-term software development. YARP was created for applications that are real-time, computation-intensive and involve interfacing with diverse and changing hardware [6]. The robotics platform was designed with the following things in mind:

- A robot control system consists of multiple processes. This means that the developer must be able to design the system as a set of processes running on a set of computers. This maximizes the amount of time spent researching instead of optimizing code. Consequently, writing and running processes has to be as easy as possible.
- Whenever we have a single process with a single responsibility, we can reuse these processes and thus have a modular process, which can be used multiple times within the same system.
- Since the actual performance of a robot controller depends on the timing of various signals, existing processes should feel minimal interference when another process is created within the system.
- Since some processes could depend on other processes, it would be ideal to minimize these dependencies to prevent expensive chains or process restarts.
- Supporting libraries should be able to be used cross-platform and external libraries should be able to be used within a YARP system, independent of YARP.
- YARP should be OS independent. Sometimes some tools are better in a one OS rather than another. YARP uses an open source library that provides a concurrent framework across a wide variety of operating systems [6].

YARP processes communicate with each other by making use of the Observer pattern and port objects. Port objects, not to be confused with actual ports on a computer, are active objects

that manage multiple connections. Each connection has a state that can be manipulated from external commands, which manage the connection or obtain state information from it. Ports can either be declared as an input, or as an output. Both input and output ports can receive and send, from and to different protocols respectively. YARP supports the following protocols:

- TCP, the reliable protocol, often used in web browsers. TCP guarantees delivery.
- UDP, a faster protocol than TCP but does not have delivery guarantee.
- Multicast, used for one-to-many connections. This is efficient when wanting to share the same information across many processes.
- Shared memory. This is used for local connections when possible.
- QNet, A fast and synchronous protocol.

YARP is flexible in the sense that it allows the programmer to either declare the port connections programmatically or at runtime. By default, ports are made to process updates and send updates often, as it is more important to be up-to-date with the information, than to process all the incoming or outgoing information. This behaviour can be altered but will introduce an overhead, similar as to UDP vs TCP.

The underlying transport layer of ports are sockets, where an initial handshake is done to agree on the exchanged datatype as well as registration. As YARP is provided in C++, all non-pointer datatypes can easily be transmitted through ports. In case that the developer does have pointer based datatypes that must be transmitted, then the developer is responsible for providing serialization and deserialization functions.

Since YARP is primarily focused on humanoid robots, it provides libraries for image processing and device drivers, as missing these are unthinkable within the humanoid domain. Unfortunately, like OROCOS, YARP does not support microcontrollers thus disqualifying this robotics platform for this thesis. Like ~~OROCOS, it ... prove to be useful if the domain of RP would extend to the domain where YARP is used.~~

2.4 Tierless Programming

MOTIVATION:

Right now we have compared some RP languages, ~~more specifically, some that run on microcontrollers~~. We have compared some robotics platforms, whose goal is to aid in robotics development. ~~However~~ robotics systems are often distributed, some parts are responsible for sensing the environment, some are responsible for reacting to it, with others responsible for everything in between. This would mean that ~~we have to create different RP programs, and somehow connect them as they are deployed~~. This seems rather inefficient, and to elaborate further we would like to introduce the concept of tierless programming or multitier programming [21].

The goal is simple. ~~We have different parts of a program that~~ live separate on different computers yet are coupled by a network. In traditional tierless systems, such as a web

environment, a developer would have to not only develop client-side code as well server side code, but also code glueing these tiers together. This gives the illusion that these tiers are decoupled from each other while in actuality they work closely together and actually also depend on each other. To avoid writing extra glue code or huge API boilerplates we could instead unify these tiers into a single source language.

Writing a tierless program in a single source language not only saves the developer from writing a lot of boilerplate code or glue code, it also abstracts away the low-level details relevant to the distributed program. This allows the developer to focus purely on the high level interactions and not worry about error-prone aspects like network communication. Additionally, it saves the developer from having to design inter-tier APIs, as the program is already inherently intertwined in the source file. How the tiers are connected becomes a compilation step rather than a development step.

In traditional distributed programming, the boundaries between hosts or functionalities may not always coincide. Meaning a single functionality may be distributed across many places and many functionalities could be isolated to a single place. Whenever functionalities are split across different devices, changing them could become cumbersome as the same functionality is implemented multiple times. Using tierless programming would allow the developer to implement it once and distribute it where it is needed. Furthermore, an update to the same functionality would also update all the functionalities that are distributed in the system.

Reasoning about a distributed system is complex. You could visualize a distributed system as a graph, where the edges between the nodes would show inter-tier communication. These edges could show that the system has a series of one-to-one, one-to-many or many-to-many relationships. Reasoning about all these different codebases or systems could become cumbersome as the edges increase and thus also the complexity. With tierless programming, you would not have to context switch between these systems or codebases as the entire system or components that compose the system are declared in a single source. The same goes for code comprehension. It is a lot easier when everything is grouped together instead of scattered.

Tierless programming languages can be categorized under the following categories:

- Degrees of tierless programming
- Placement strategy
- Placement specification & granularity
- Communication abstractions
- Supported topologies.

1. Degree of multitier programming

Tierless programming languages either provide support for explicit placement of distribution by means of for example an annotation such as `@server` or `@client`, or they split code automatically by means of static analysis. Alternatively, some tierless languages translate code across target platforms (eg: Java to JavaScript).

2. Placement strategy

The placement strategy is the strategy adopted by the tierless language to split the program up. A Tierless program can be partitioned into different programs (statically or dynamically). Another option would be to separate the program into different stages, where the execution of one stage generates the next stage and can inject values computed in the current stage into the next.

3. Placement specification & Granularity

The tierless languages that allow for explicit placement, can optionally also allow varying degrees in granularity of the placement. Some tierless languages could for example only allow complete blocks to be placed together on a tier whilst other languages allow local bindings to be placed on different tiers.

4. Communication abstractions

Tierless languages also differ on the underlying communication used, so could one language use Remote Procedure Calls (RPC), asynchronous callbacks, directional message-passing channels, pub-sub mechanisms, distributed shared state or even FRP streams.

5. Supported topologies

Another categorizing property of tierless programming is their supported topologies. Most web-oriented tierless languages focus on the client-server architecture. This means that most of the time they're exclusively meant for that architecture, and sometimes also allow integration of a database. Some tierless languages allow specification of tiers, such as Scalaloci [22].

```
@multitier object Chat {
    @peer type Server <: { type Tie <: Multiple[Client] }
    @peer type Client <: { type Tie <: Single[Server] }

    val message = on[Client] { Evt[String] }

    val publicMessage = on[Server] {
        message.asLocalFromAllSeq map { case (_, message) ⇒ message }
    }

    def main() on[Client] {
        publicMessage.asLocal observe println
        for (line ← io.Source.stdin.getLines)
           message.fire(line)
    }
}
```

Listing 11: Tierless program example from the ScalaLoci tutorial

To give an example how such a tierless program is written, consider the following example in ScalaLoci defined in Listing 11. It consists of a server and multiple clients where message events are sent to the server, who then makes clients print it to their consoles. By using `@peer` annotations we can define the structure of the tierless program. We can define whether there is multiple of a tier or if there is a single one, like multiple clients and a single server in this case. The cross-tier boundaries are generated by the compiler as needed.

2.5 Problem Statement

Right now we have presented a collection of RP languages, robotics platforms and the concept of tierless programming. What we want to achieve with this thesis is to be able to create an RP program meant for a distributed robotics network consisting of at least a single computer with an OS as well as microcontrollers. Furthermore, we want to achieve this making use of Haai together with a popular robotics platform, which to our knowledge does not exist yet. In this thesis we will choose ROS as our robotics platform as it can interop with other robotics platforms. Distributing the RP program across different devices can be aided using tierless programming.

To give list of steps how we want to achieve our goal, we initially would have to extend the Haai language to provide abstractions and syntax for tierless programming. Secondly, we would need to create a tool that splits the program up in tiers, with some additional metadata which would help in creating the boundary related code. Afterwards we would have to generate the tier-related code compatible with in this case ROS 2 and micro-ROS, depending on the tier. Finally, since Haai is compiled down to bytecode we would require compatible interpreters for both tiers that also interop with ROS 2 or micro-ROS. For the ROS 2 side we could use the existing Remus VM, and a foreign function interface (ffi) which allows usage of the ROS 2 libraries from another language. For micro-ROS we would have to create a new VM in a

low-level language that is compatible with micro-ROS.